Course: Game Theory
Programming Language: Python
Assessment Type: Semester Rolling Project
Project Title: Design and Implementation of a Game-Theoretic Analysis System Using Python

1. Project Description

Students are required to identify a strategic decision-making problem and develop a Python-based application to model, analyse, and interpret the interactions between decision-makers using game theory.

Students will apply game theory concepts and Python programming techniques to represent players, strategies, payoffs, and possible outcomes. The application should perform appropriate strategic analyses and present the results in a clear and understandable format.

Students may select a problem from economics, business, cybersecurity, politics, environmental management, computer science, or another relevant field involving strategic interactions.

2. Project Objectives

By the end of the project, students should be able to:

1. Explain fundamental concepts and principles of game theory.
2. Formulate strategic decision-making problems as mathematical models.
3. Represent games using payoff matrices and game trees.
4. Implement game-theoretic models using Python.
5. Identify dominant and dominated strategies where applicable.
6. Calculate best responses and identify pure-strategy Nash equilibria.
7. Analyse mixed strategies and sequential games where appropriate.
8. Use Python libraries to perform calculations and visualize results.
9. Interpret computational results and explain their practical implications.
10. Document, test, and present a working game-theoretic analysis application.

3. Project Requirements

The Python application must demonstrate appropriate use of the following concepts:

1. **Problem Formulation:** Define the strategic problem, players, objectives, assumptions, and rules.
2. **Strategy Representation:** Represent the available strategies of each player.
3. **Payoff Matrices:** Store and display the payoffs associated with different strategy combinations.
4. **Python Fundamentals:** Use variables, data types, conditional statements, loops, functions, and appropriate data structures.
5. **Strategic Analysis:** Implement calculations for best responses and dominant strategies where they exist.
6. **Nash Equilibrium:** Identify pure-strategy Nash equilibria for suitable finite games.
7. **Mixed Strategies:** Implement mixed-strategy calculations for suitable games where pure-strategy analysis is insufficient.
8. **Visualization:** Present payoff matrices, strategy comparisons, or game trees using appropriate visualization tools.

9. **Scenario Analysis:** Examine how changing payoff values affects strategic outcomes.
10. **Testing and Validation:** Verify calculations using known examples and test cases.
11. **User Interface:** Provide a command-line interface, notebook interface, or graphical interface for entering and analysing games.
12. **Documentation:** Explain the mathematical model, Python implementation, results, and limitations.

4. Semester Rolling Project Stages

Stage 1: Problem Identification and Proposal

- Identify a strategic decision-making problem.
- Define the project objectives and research questions.
- Identify the players and their interests.
- Explain why game theory is appropriate.
- Specify the intended Python application.

Deliverable: Project proposal and problem specification.

Stage 2: Introduction to Python and Game Representation

- Set up the Python environment.
- Represent players and strategies using lists or other suitable data structures.
- Create and display payoff matrices using Python.
- Implement basic input and output operations.

Deliverable: A Python program that represents and displays a simple game.

Stage 3: Payoff Matrix Analysis

- Accept or define payoff values.
- Store payoffs using lists, nested lists, NumPy arrays, or pandas DataFrames.
- Compare the payoffs associated with different strategies.
- Identify the best response of each player.

Deliverable: A working payoff matrix analysis module.

Stage 4: Dominant Strategies and Nash Equilibrium

- Implement procedures for identifying dominant strategies.
- Identify dominated strategies where appropriate.
- Develop an algorithm to identify pure-strategy Nash equilibria in finite games.
- Test the algorithm using suitable examples.
- Explain the results in game-theoretic terms.

Deliverable: A Python program that analyses dominant strategies and pure-strategy Nash equilibria.

Stage 5: Advanced Game-Theoretic Analysis

- Implement mixed-strategy calculations for appropriate games.

- Represent sequential games using game trees where relevant.
- Analyse repeated or cooperative interactions where appropriate.
- Compare the outcomes of different strategic models.

Deliverable: An extended game analysis module.

Stage 6: Visualization and Scenario Analysis

- Visualize payoff matrices and strategic outcomes.
- Use suitable Python libraries to generate charts or diagrams.
- Change payoff values and recalculate the results.
- Compare equilibrium outcomes across different scenarios.

Deliverable: Visualizations and a scenario analysis report.

Stage 7: System Integration and Testing

- Integrate the developed Python modules.
- Test input validation and numerical calculations.
- Compare program results with manually calculated examples.
- Identify and correct errors.
- Improve the usability of the application.

Deliverable: Tested Python application and test report.

Stage 8: Final Report and Presentation

- Complete the application and its documentation.
- Explain the mathematical foundations and algorithms.
- Present sample games, calculations, and results.
- Discuss limitations and possible improvements.
- Demonstrate the application and defend the findings.

Deliverable: Complete source code, final project report, and presentation.

5. Suggested Python Libraries

Students may use the following libraries where appropriate:

Library	**Purpose**
Python standard library	Core programming, functions, and data structures
NumPy	Numerical computations and payoff matrix operations
pandas	Organizing and displaying payoff data
Matplotlib	Plotting and visualizing results
NetworkX	Representing and visualizing game trees
Jupyter Notebook	Combining code, mathematical explanations, and results
Nashpy	Analysing selected two-player games, including Nash equilibria

Students should implement the core algorithms themselves where required by the lecturer. External libraries may be used for comparison and validation.

6. Expected Project Deliverables

Each student or group must submit:

1. Project proposal and problem specification.
2. Mathematical formulation of the selected game.
3. Definition of players, strategies, assumptions, and payoffs.
4. Python source code.
5. Payoff matrix or game tree.
6. Implementation of the required game-theoretic algorithms.
7. Sample outputs and visualizations.
8. Test cases and validation results.
9. Final project report.
10. Presentation and demonstration of the application.

7. Submission Guidelines

- The project may be completed individually or in groups.
- Python must be used as the primary implementation language.
- All mathematical assumptions and payoff values must be clearly explained.
- Students must be able to explain their algorithms and calculations.
- Code must be properly organized, readable, and documented.
- All calculations must be tested against appropriate examples.
- All external sources and libraries must be acknowledged.
- A video link on the google drive should be submitted entailing your explanation of the project.